Numbers and Representations - Part 1

K. Sridharan

Review

- In the introductory class, we saw what a digital system is.
- We also indicated that digital does not mean binary and one could have alternate ways of representing numbers.

The Binary Number System

- One can have unsigned and signed numbers in binary format.
- We will see what these mean in the slides to follow.

Unsigned Binary Number

- Let us say it has n-bits running from x_0 (LSB) to x_{n-1}
- Then, its value is given by

$$x = x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- **Range: 0 to $+2^n - 1$**

Unsigned vs. Signed

- Note that while addition of unsigned numbers will lead to an unsigned number, subtraction does not necessarily.
- Computer programs calculate positive and negative numbers so we need a representation that makes a distinction between positive and negative numbers.

Signed Numbers – General Information

- To handle signed numbers, need to identify an appropriate representation
- Three ‘standard’ methods for signed number representation exist (for binary numbers): (i) Sign-Magnitude representation (ii) 1’s complement representation and (iii) 2’s complement representation

Sign-Magnitude representation

- Consists of sign and magnitude parts; the MSB is devoted to sign: 0 in MSB denotes that the number is positive while 1 in MSB means that the number is negative
- Example: 0101 corresponds to 5 while 1110 is -6

What is a limitation of the Sign-Magnitude format ?

- Question for students

Limitation of Sign-Magnitude format

- Non-unique representation for zero (1000 as well 0000 represent 0).

Complement formats

- The non-uniqueness of representation for zero leads to investigations on other ways to represent signed numbers
- 1’s and 2’s complement representations are appropriate for binary numbers.

Finding 1's complement representation

- Given an integer, the 1's complement representation is the same as the binary representation if the number is positive
- If the integer is negative, we get the 1's complement representation by writing out the binary representation for the positive (counterpart) and then flipping the bits

What is the drawback of 1’s complement representation ?

- Question for students

Drawback of 1's complement representation

- Once again, non-unique representation for 0

2’s complement representation

- This is the same as the binary representation if the number is positive.
- If the number is negative, we write the binary representation for the positive (counterpart), flip each bit and then add 1

How about representation for zero ?

- Question for students

Unique representation for zero

- Note that we can represent 0 uniquely in 2's complement (and do not have the problem faced with sign-magnitude or 1's complement)

Another advantage with 2's complement representation

- 2's complement addition can be implemented by an adder designed for unsigned numbers.
- Example: Suppose we want to add -5 and -3 given their 2's complement representations, namely 1011 and 1101 respectively. Now simply add (as though you are adding unsigned numbers). The result is 11000 which corresponds to -8

Verifying the result

- How does 11000 correspond to -8 ?
- One way is to see how +8 would be and generate its negative counterpart.
- Note that +8 is given by 01000. Now complement each bit and add 1.

Why do we need to prefix by 0 ?

- We have said that the representation of 8 (in 2’s complement format) is 01000.
- Why should we have a 0 in front of 1 ?

Why do we need to prefix by 0 ?

- Note that we are dealing with signed numbers so need to prefix by 0 to get the representation for +8

What about subtraction in 2's complement ?

- Yes, 2's complement subtraction can also be implemented by a simple adder for unsigned numbers.
- Example: Subtract -2 from 5

The 2's complement representations are 1110 and 0101. Take 2's complement of 1110 which is 0010. Use an adder for unsigned numbers to add 0010 with 0101 to get 0111

Textbook and Reference

- M. Mano and M. Ciletti, Digital Design, Pearson Education, 2018
- C. Roth, Fundamentals of Logic Design, Thomson Brooks/Cole, 2005